

Infosys Springboard Virtual Internship 7.0

Completion Report

Team Details

Batch Number: Group 1_Batch-1

Start date: 29th June 2026

Name: Chandru R

Internship Duration: 8 Weeks

COMPLETION / PROJECT REPORT

MediSphere — Healthcare Management Platform

Project Detail	Information
Name	Chandru R
Project Title	MediSphere — Healthcare Management Platform
Internship	Infosys Springboard Virtual Internship 7.0
Duration	8 Weeks
Start Date	29 June 2026
Frontend	React + Vite
Backend	Java 21 + Spring Boot
Database	PostgreSQL (Neon)
Security	Spring Security + JWT + BCrypt
Backend Deployment	Render

1. Project Title

MediSphere — Healthcare Management Platform

MediSphere is a full-stack healthcare management application designed to connect administrators, doctors, and patients through a secure, role-based workflow. The platform centralizes patient management, doctor management, appointment scheduling, and prescription management in one system.

2. Project Objective

The primary objective of MediSphere is to provide a secure and user-friendly platform for managing common hospital operations digitally. The system reduces manual coordination between patients, doctors, and administrators by providing dedicated workflows and protected REST APIs.

- Provide secure login and role-based access for Admin, Doctor, and Patient users.
- Allow administrators to manage doctors, patients, appointments, and prescriptions.
- Allow doctors to manage their appointments and create or update prescriptions.
- Allow patients to register/login, book appointments, track appointment status, and view their prescriptions.

- Maintain healthcare data in PostgreSQL with JPA-based persistence.
- Send important patient-facing email notifications through SMTP.
- Provide a responsive React frontend connected to the Spring Boot REST backend.

3. Project Description in Detail

MediSphere follows a layered full-stack architecture. The React/Vite frontend communicates with a Java 21 Spring Boot backend through REST APIs. Spring Security and JWT protect the APIs, while role-based authorization controls which operations can be performed by each user type. PostgreSQL stores users, doctors, patients, appointments, and prescriptions.

The backend is organized into controllers, services, repositories, entities, DTOs, security components, validation, and exception handling. The frontend provides separate user experiences for Admin, Doctor, and Patient roles while using the same backend API.

The deployed backend is hosted on Render and the frontend can be configured to use the deployed API base URL through an environment variable.

3.1 Main Technology Stack

Layer	Technology / Tool	Purpose
Frontend	React + Vite	Responsive user interface and role-based dashboards
API	Spring Boot REST	Backend business logic and HTTP APIs
Language	Java 21	Backend implementation
Security	Spring Security, JWT, BCrypt	Authentication, authorization and password hashing
Persistence	Spring Data JPA / Hibernate	Database access and ORM
Database	PostgreSQL / Neon	Persistent healthcare data
Email	SMTP / Mail service	Registration, appointment and prescription email notifications
Build	Maven / npm	Backend and frontend builds
Deployment	Render / Vercel	Application hosting
Testing	Postman / Swagger	REST API testing and verification

3.2 End-to-End Workflow

The overall workflow is designed around three roles:

- Admin → logs in → manages doctors and patients → monitors appointments → manages prescriptions.
- Patient → registers/logs in → selects a doctor → books an appointment → waits for approval → tracks the appointment → receives/views prescription.
- Doctor → logs in → views assigned appointments → approves/completes or manages appointment workflow → creates/updates a prescription for an eligible appointment → patient can view the prescription.

A prescription is linked one-to-one with an appointment. This keeps the clinical record connected to the consultation that produced it. Appointment deletion is configured to remove its dependent prescription so the database does not retain an orphaned prescription record.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Requirements & architecture	Defined Admin, Doctor, Patient roles; designed modules and API workflow
Week 2	Authentication	Spring Boot setup, PostgreSQL connection, JWT login and protected routes
Week 3	Doctor management	Doctor CRUD, profile details, availability, consultation fee and role-based access
Week 4	Patient management	Patient registration/login and patient data management
Week 5	Appointment workflow	Book, view, update status, cancel/delete appointments
Week 6	Prescription workflow	Create, view, update and delete prescriptions linked to appointments
Week 7	Frontend integration	Admin, Doctor and Patient dashboards connected to backend APIs
Week 8	Testing & deployment	Postman/Swagger testing, validation, bug fixing and deployment preparation

5a. Key Milestones

Milestone	Description	Achieved
Project Kickoff	Requirements study, architecture planning and module definition	Week 1
Authentication	JWT login, password hashing and protected role-based APIs	Week 2
Core Management	Doctor and patient management workflows	Weeks 3–4
Appointment Module	Booking, status management and appointment lifecycle	Week 5
Prescription Module	Prescription creation, update, retrieval and appointment linkage	Week 6
Frontend Integration	Role-specific dashboards and API integration	Week 7
Testing & Deployment	API testing, fixes, environment configuration and deployment	Week 8

5b. Project Execution Details

Requirement analysis: The healthcare workflow was divided into role-specific operations so that each user can access only the functions relevant to the role.

Backend development: REST APIs were implemented using Java 21, Spring Boot, Spring Data JPA, PostgreSQL, Spring Security, JWT, BCrypt, validation and service/repository layers.

Frontend development: React/Vite pages were connected to the REST API using Axios. Separate Admin, Doctor and Patient areas were implemented with protected navigation and role-aware screens.

Communication: SMTP email support was integrated for important patient-facing notifications, including password/OTP-related flows and prescription or appointment communication where configured.

Testing: Endpoints were tested through Postman and Swagger/OpenAPI. Deployment logs and browser/API responses were used to identify and correct integration issues.

6. Snapshots / Screenshots

The following screenshots are from the actual MediSphere project and demonstrate the major user-facing workflows.

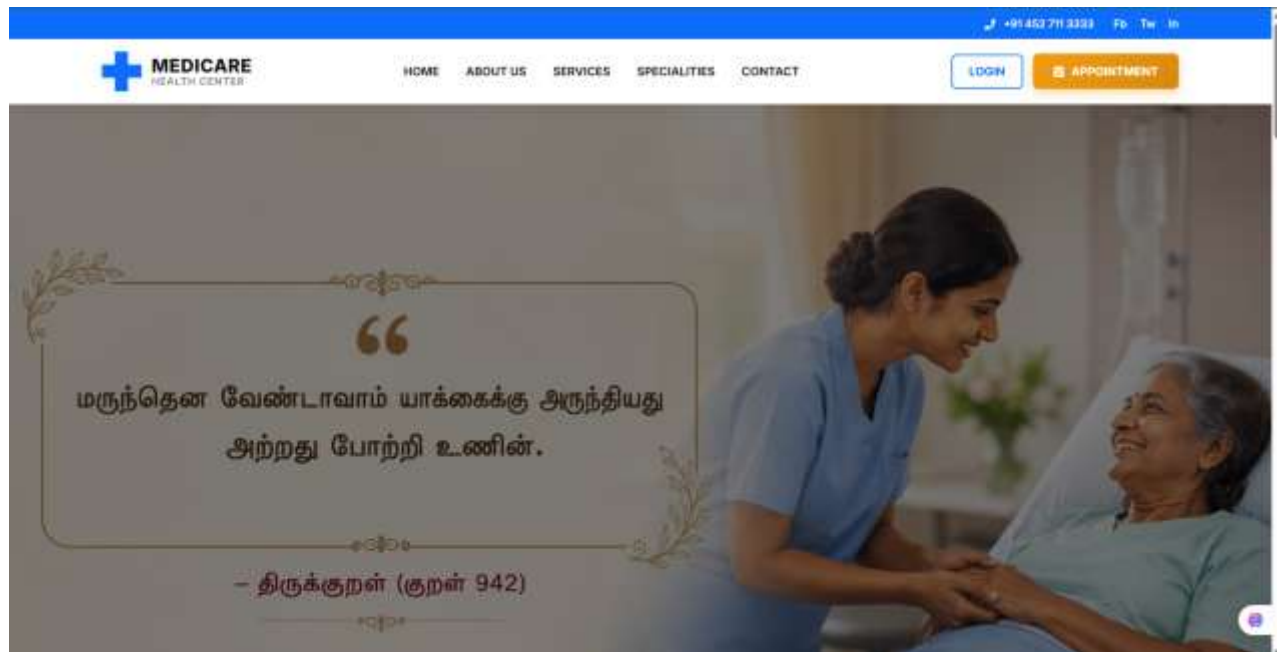
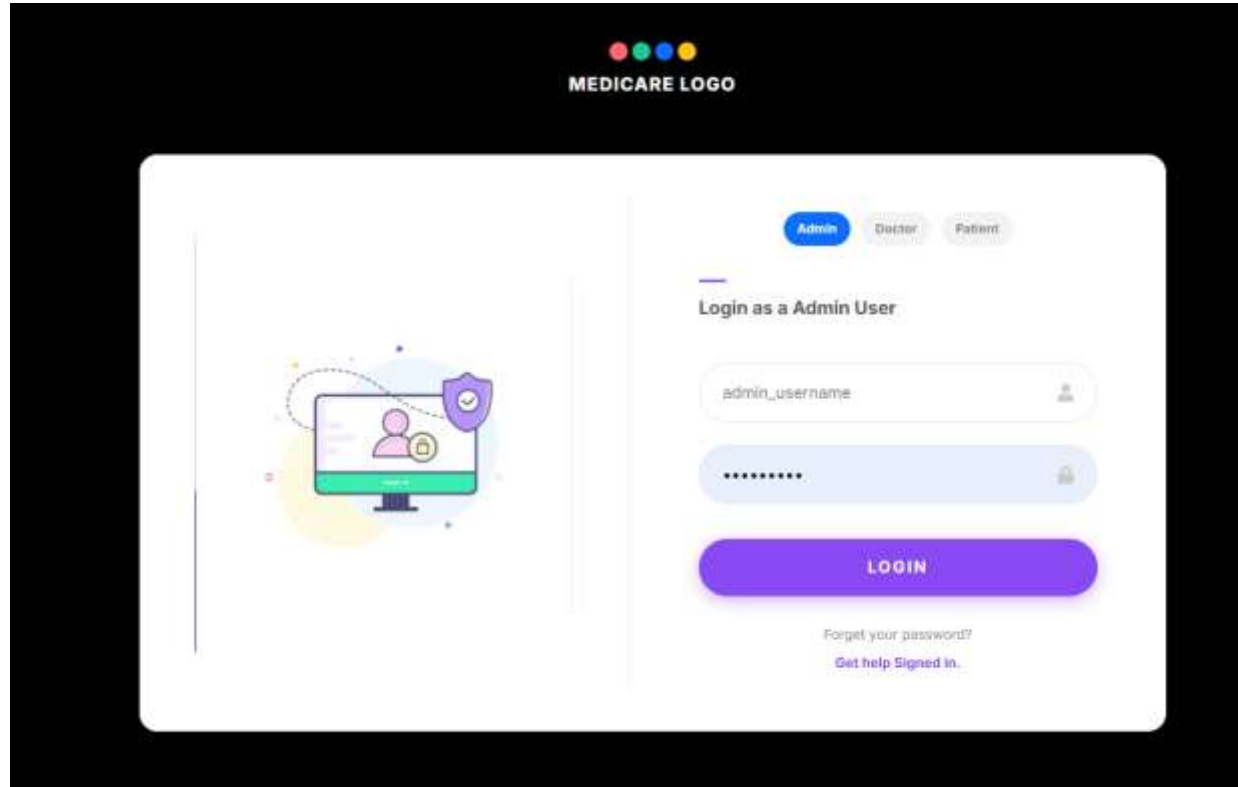


Figure 1. MediSphere healthcare landing page.

Figure 2. MediSphere role-based login interface for Admin, Doctor and Patient.

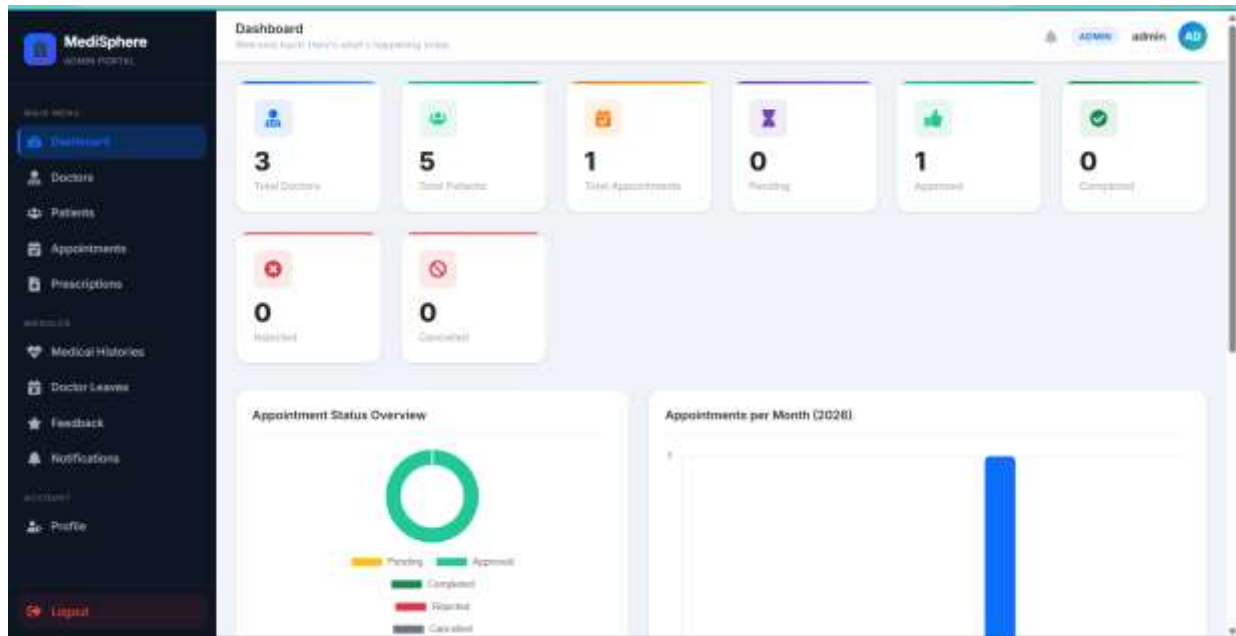


Figure 3. Admin dashboard showing doctors, patients and appointment status statistics.

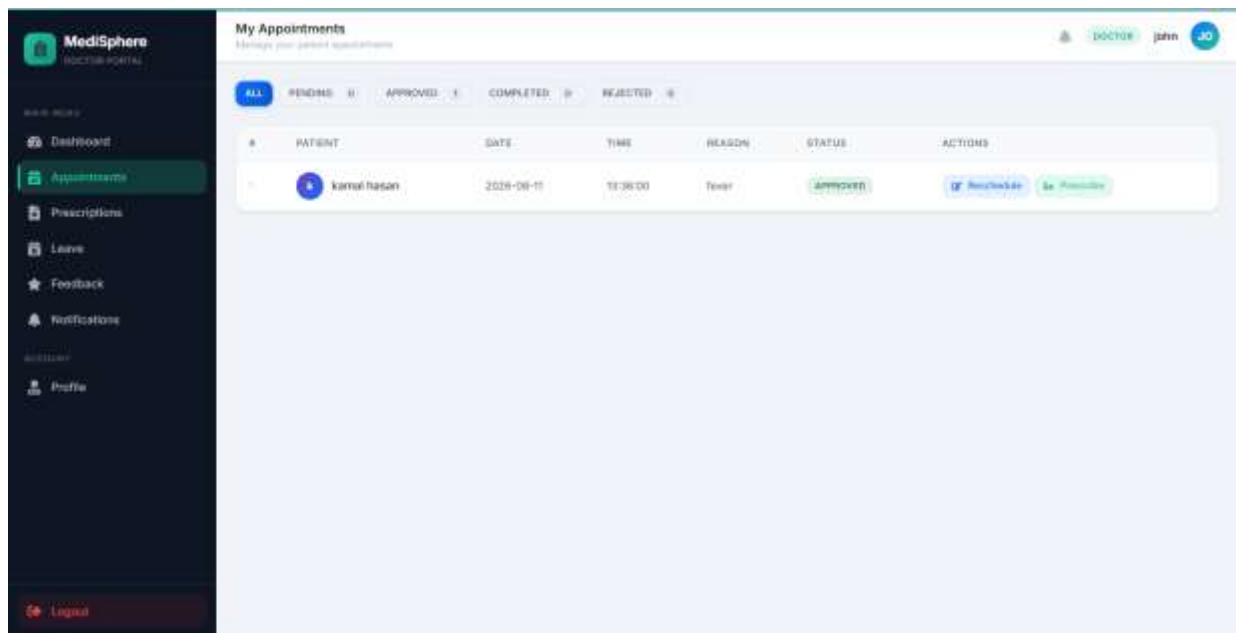


Figure 4. Doctor appointment dashboard showing patient appointment and prescription action.

GET	/doctors/{id}
PUT	/doctors/{id}
DELETE	/doctors/{id}
GET	/doctors
POST	/doctors
POST	/doctors/login
GET	/doctors/search/specialization
GET	/doctors/search/name
GET	/doctors/profile
GET	/doctors/page
GET	/doctors/available

Book Appointment
Choose a doctor and schedule your visit

PATIENT
kamal hasan
KA

Home — Appointment

Appointment

Online Booking

Make an Appointment

Select Doctor
Dr. Priya Sharma – Dermatology

Date
24-08-2026

Preferred Time
16:41

Your Message
foveo

Why Us

Why Choose Us

Medicare Health Center, the No.1 Multi speciality Hospital in South Tamil Nadu, providing exceptional care around the clock.

Doctors on Duty
450+ doctors available round the clock

Happy Patients
40 Lakh+ Happy Patients

Nursing Staff
1100+ Dedicated Nursing Staff

Figure 5. Patient appointment booking interface with doctor, date, time and reason.

Figure 6. REST API documentation showing appointment and authentication endpoints.

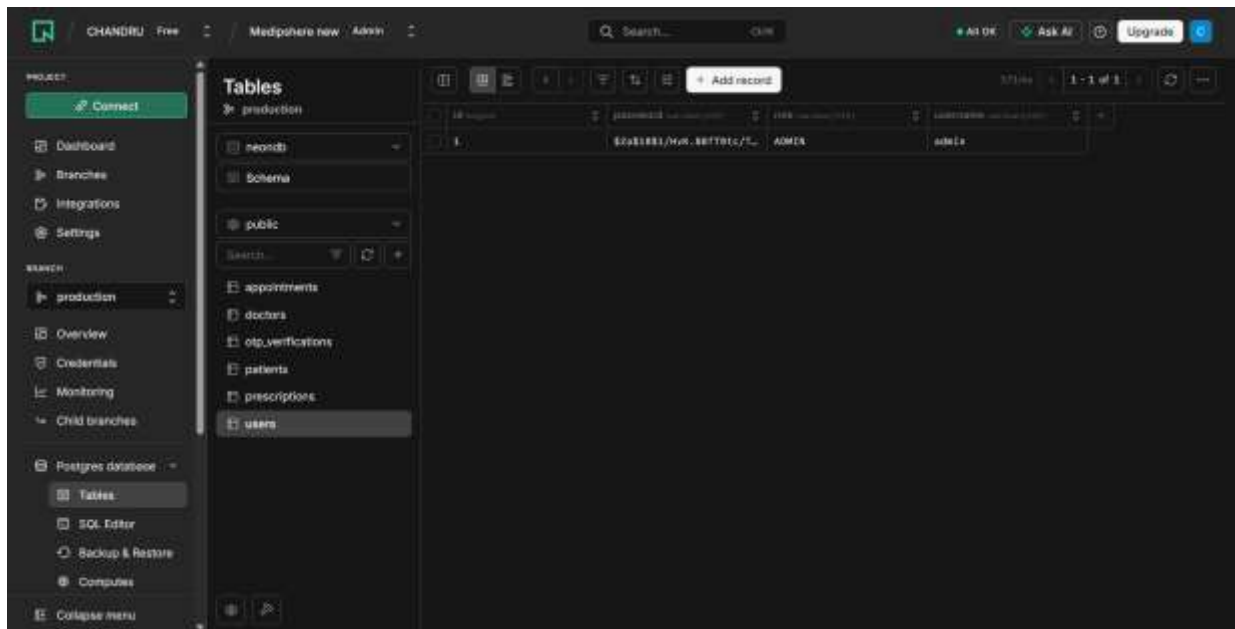


Figure 7. PostgreSQL/Neon database tables used by MediSphere.

6.1 Role-Based Functional Workflow

Admin

- Secure login using Admin credentials.
- View dashboard statistics for doctors, patients and appointments.
- Add, view, update and delete doctors.
- Manage doctor details such as specialization, qualification, experience, availability, consultation fee and status.
- Add, view, update and delete patients.
- View and manage appointments.
- View all prescriptions and delete prescriptions when required.
- Access protected administrative APIs through JWT + role authorization.

Doctor

- Secure Doctor login using email/username and password.
- View appointments assigned to the doctor.
- Review appointment information and manage the appointment workflow.
- Create a prescription only for an approved or completed appointment.
- Update an existing prescription.
- View prescription details allowed by the API.
- Prescription data is linked to the corresponding appointment and patient.

Patient

- Register and log in using patient credentials.
- View available doctors and their professional/availability details.
- Book an appointment by selecting doctor, date, time and reason.
- View appointment status and appointment information.
- Access personal prescriptions through the protected /prescriptions/my workflow.
- Receive configured email notifications for relevant account/appointment/prescription events.

6.2 Important Backend API Workflow

Module	Typical Operations	Role
Authentication	Login / JWT token / protected requests	Admin / Doctor / Patient
Doctors	Add / View / Update / Delete	Admin
Patients	Register / Login / View / Update / Delete	Admin / Patient (role-dependent)
Appointments	Book / View / Update status / Cancel or Delete	Admin / Doctor / Patient
Prescriptions	Create / View / Update / Delete / My prescriptions	Admin / Doctor / Patient (read own)
Email	SMTP notifications / password-related flows	System service

Example patient prescription flow: Patient logs in → JWT is stored by the frontend → patient requests GET /prescriptions/my → Spring Security verifies the PATIENT role → the backend uses the authenticated email to find prescriptions linked to the patient's appointments → prescription DTOs are returned to the frontend.

7. Challenges Faced

- Configuring secure JWT authentication and role-based authorization for three user roles.
- Maintaining consistent entity relationships between Doctor, Patient, Appointment and Prescription.
- Handling database foreign-key constraints when deleting appointments that have prescriptions.
- Validating request payloads and handling invalid enum/boolean values.
- Configuring PostgreSQL/Neon and environment-based deployment settings.
- Connecting the React frontend to the deployed Spring Boot backend.
- Configuring SMTP/email services and environment variables safely.
- Debugging deployment and browser/API errors using server logs, Postman and Swagger.

8. Learnings & Skills Acquired

- Java 21 and Spring Boot application development.
- REST API design using Spring MVC.
- Spring Data JPA, Hibernate and relational database mapping.
- Spring Security, JWT authentication, BCrypt password hashing and RBAC.
- DTO-based request/response design and validation.
- React/Vite frontend development and Axios API integration.
- PostgreSQL database management and cloud database usage with Neon.
- SMTP/email integration and environment-based configuration.

- API testing using Postman and Swagger/OpenAPI.
- Git/GitHub workflow, deployment debugging and full-stack integration.

9. Project Outcome

MediSphere provides a working end-to-end healthcare management workflow in which Admin, Doctor and Patient users interact with the same secure backend according to their roles. The system connects authentication, doctor and patient management, appointment scheduling, and prescription management into a single application.

10. Conclusion

The MediSphere project provided practical experience in developing and integrating a full-stack application using modern Java and React technologies. The project demonstrates how secure REST APIs, relational data modeling, role-based access control and a responsive frontend can be combined to support a realistic healthcare workflow. The modular architecture also leaves room for future additions such as billing, richer medical history, analytics and notification enhancements.

11. Acknowledgements

I sincerely thank Infosys Springboard and the internship mentors for their guidance, feedback and support throughout the project. I also appreciate everyone who contributed feedback during development and testing. This project significantly strengthened my practical skills in full-stack development, backend security, database management, API integration and deployment.

Appendix A — Core Data Relationship

Doctor 1 —< Appointment >— 1 Patient

Appointment 1 — 1 Prescription

This relationship ensures that a prescription belongs to a specific consultation/appointment and can be retrieved for the patient through the appointment relationship.

Appendix B — Security Flow

- User submits login credentials.
- Backend verifies the user and BCrypt password hash.
- Backend generates a JWT containing the authenticated identity/role.
- Frontend sends the token in the Authorization header for protected API calls.
- Spring Security validates the token and role before the controller method executes.
- The service layer performs the business operation and returns a DTO response.